CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Experiment – 2

Name: Prionti Das
UID: 24BAI71005
Branch: CSE(AIML)
Section/Group: 24AIT_NTPP-3(B)
Semester: 5th
Date of Performance: 09/07/2026
Subject: Full Stack
Subject Code: 24CSP-337

EXPERIMENT – 2.1

1. AIM:

To design and implement a centralized state management system using Redux Toolkit for managing posts and platform-related data.

2. OBJECTIVE:

1. To understand the concept of global state management
2. To implement Redux Toolkit for scalable state handling
3. To design a normalized state structure
4. To manage asynchronous data flows (optionally via mock APIs)

3. SOFTWARE REQUIREMENTS:

1. Node.js & npm
2. React.js
3. Redux Toolkit
4. React-Redux
5. Code Editor (VS Code)

4. THEORY:

As frontend applications grow in complexity, managing state across multiple components becomes increasingly challenging. Traditional approaches such as prop drilling lead to tightly coupled components and reduced maintainability.

Redux provides a centralized store that acts as a **single source of truth**, enabling predictable state transitions through actions and reducers. Redux Toolkit simplifies Redux implementation by reducing boilerplate and providing built-in utilities for common tasks.

State normalization is an important concept where data is stored in a structured format (similar to relational databases), reducing redundancy and improving performance. This is especially useful when managing collections such as posts or users.
Additionally, Redux supports handling asynchronous operations, enabling integration with APIs or simulated mock data sources.

5. IMPLEMENTATION/PROCEDURE:

1. Install Redux Toolkit and React-Redux
2. Configure Redux store
3. Create slices for posts and platforms
4. Define initial state structure
5. Implement reducers for CRUD operations
6. Connect components to Redux store using hooks
7. Integrate mock API calls using async thunks

6. CODE :

```
import React from "react";
import { Provider, useDispatch, useSelector } from "react-redux";
import { configureStore, createSlice } from "@reduxjs/toolkit";

const postSlice = createSlice({
  name: "post",

  initialState: {
    platform: "LinkedIn",
    post: "",
    savedPosts: [],
    likes: 0,
  },

  reducers: {
    setPlatform: (state, action) => {
      state.platform = action.payload;
    },

    setPost: (state, action) => {
      state.post = action.payload;
    },

    savePost: (state) => {
      if (state.post.trim() !== "") {
        state.savedPosts.push({
          platform: state.platform,
          content: state.post,
          date: new Date().toLocaleString(),
        });

        state.post = "";
      }
    },

    clearPost: (state) => {
      state.post = "";
    },
```

```
    likePost: (state) => {
      state.likes += 1;
    },
  },
});

const {
  setPlatform,
  setPost,
  savePost,
  clearPost,
  likePost,
} = postSlice.actions;

const store = configureStore({
  reducer: {
    post: postSlice.reducer,
  },
});

function PostComposer() {
  const dispatch = useDispatch();

  const { platform, post, savedPosts, likes } = useSelector(
    (state) => state.post
  );

  const limits = {
    Twitter: 280,
    LinkedIn: 3000,
    Instagram: 2200,
  };

  const limit = limits[platform];
  const isExceeded = post.length > limit;

  return (
    <div
      style={{
        backgroundColor: "#f4f6f9",
        minHeight: "100vh",
        padding: "40px",
        fontFamily: "Arial, sans-serif",
      }}
    >
      <div
        style={{
          width: "700px",
          margin: "auto",
          backgroundColor: "#ffffff",
          padding: "30px",
          borderRadius: "15px",
          boxShadow: "0px 4px 15px rgba(0,0,0,0.2)",
        }}
      >
```

```
<h1
  style={{
    textAlign: "center",
    color: "#0A66C2",
    marginBottom: "30px",
  }}
>
  Redux Toolkit Post Composer
</h1>

<label>
  <b>Select Platform</b>
</label>

<br />
<br />

<select
  value={platform}
  onChange={(e) => dispatch(setPlatform(e.target.value))}
  style={{
    width: "100%",
    padding: "10px",
    borderRadius: "8px",
    fontSize: "16px",
  }}
>
  <option>Twitter</option>
  <option>LinkedIn</option>
  <option>Instagram</option>
</select>

<br />
<br />

<label>
  <b>Write Your Post</b>
</label>

<br />
<br />

<textarea
  rows="6"
  placeholder="Type your post here..."
  value={post}
  onChange={(e) => dispatch(setPost(e.target.value))}
  style={{
    width: "100%",
    padding: "10px",
    borderRadius: "10px",
    border: "2px solid #ccc",
    fontSize: "16px",
    resize: "none",
  }}
```

```
></textarea>

<br />
<br />

<p
  style={{
    fontWeight: "bold",
    color: isExceeded ? "red" : "#2c3e50",
  }}
>
  Characters: {post.length} / {limit}
</p>

{isExceeded ? (
  <p
    style={{
      color: "red",
      fontWeight: "bold",
    }}
  >
    Character Limit Exceeded!
  </p>
) : (
  <p
    style={{
      color: "green",
      fontWeight: "bold",
    }}
  >
    Ready to Post
  </p>
)}

<button
  onClick={() => dispatch(savePost())}
  style={{
    backgroundColor: "#007bff",
    color: "white",
    border: "none",
    padding: "12px 20px",
    borderRadius: "8px",
    fontSize: "16px",
    cursor: "pointer",
    marginRight: "10px",
  }}
>
  Save Post
</button>

<button
  onClick={() => dispatch(clearPost())}
  style={{
    backgroundColor: "#dc3545",
    color: "white",
    border: "none",
```

```
    padding: "12px 20px",
    borderRadius: "8px",
    fontSize: "16px",
    cursor: "pointer",
    marginRight: "10px",
   }}
  >
   Clear
  </button>

  <button
   onClick={() => dispatch(likePost())}
   style={{
    backgroundColor: "#28a745",
    color: "white",
    border: "none",
    padding: "12px 20px",
    borderRadius: "8px",
    fontSize: "16px",
    cursor: "pointer",
   }}
  >
   ❤️ Like
  </button>

  <p
   style={{
    fontSize: "18px",
    fontWeight: "bold",
    color: "#0A66C2",
    marginTop: "20px",
   }}
  >
   ❤️ Total Likes: {likes}
  </p>

  <hr />

  <h2 style={{ color: "#0A66C2" }}>
   Saved Posts
  </h2>

  {savedPosts.length === 0 ? (
   <div
    style={{
     padding: "15px",
     backgroundColor: "#f8f9fa",
     borderRadius: "10px",
     border: "1px solid #ccc",
    }}
   >
    No Posts Saved
   </div>
  ) : (
   savedPosts.map((item, index) => (
    <div
```

```
            key={index}
            style={{
              backgroundColor: "#f8f9fa",
              padding: "15px",
              borderRadius: "10px",
              border: "1px solid #ccc",
              marginBottom: "15px",
            }}
          >
            <h3 style={{ color: "#0A66C2", margin: 0 }}>
              {item.platform}
            </h3>

            <p>{item.content}</p>

            <small>{item.date}</small>
          </div>
        ))
      )}
    </div>
  </div>
 );
}

function App() {
 return (
  <Provider store={store}>
   <PostComposer />
  </Provider>
 );
}

export default App;
```

7. OUTPUT :

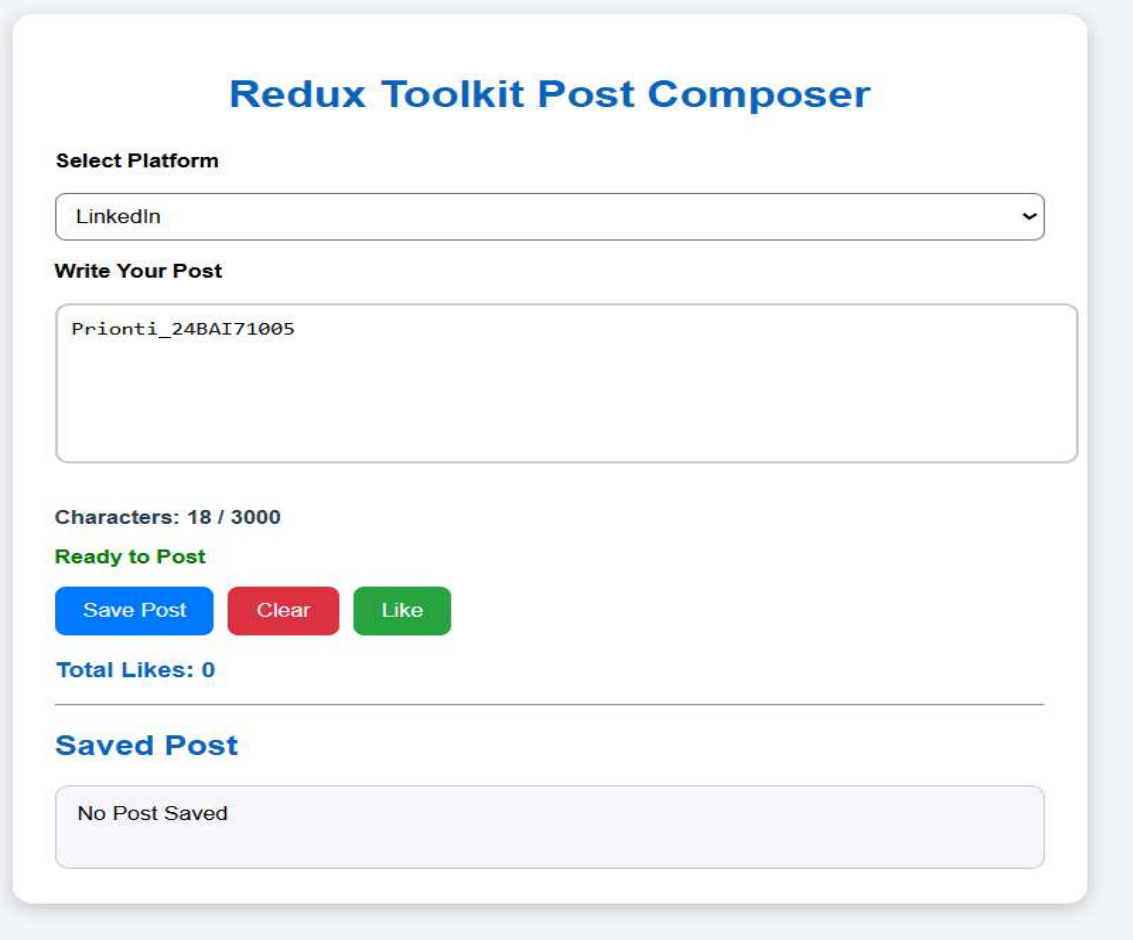


Fig 2.1: Redux Toolkit Post Composer User Interface

The **Redux Toolkit Post Composer** is a React application that allows users to create and manage social media posts. It uses **Redux Toolkit** for centralized state management to store the selected platform, post content, saved posts, and like count. The application provides **real-time character limit validation**, allows users to **save or clear posts**, and updates the **like count** efficiently using a single Redux store. It demonstrates the use of **React components, Redux Toolkit, state management, and event handling** in a simple and interactive application.

8. LEARNING OUTCOME:

- Understand the concept of **centralized state management** using Redux Toolkit
- Create and configure a **Redux Store** for managing application data
- Use **Slices, Actions, and Reducers** to update and manage application state.
- Connect React components with Redux using `useSelector()` and `useDispatch()`.
- Implement features such as **saving posts, clearing posts, liking posts, and platform selection** using Redux Toolkit.
- Build efficient and maintainable React applications by managing shared state in a centralized manner.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

EXPERIMENT – 2.2

1. AIM:

To optimize state access and improve application performance using memoized selectors and efficient rendering strategies.

2. OBJECTIVE:

1. To understand the concept of derived state
2. To implement memoized selectors using Reselect
3. To reduce unnecessary component re-renders
4. To improve performance in large-scale applications

3. SOFTWARE REQUIREMENTS:

1. React.js
2. Redux Toolkit
3. Reselect (or built-in createSelector)
4. Code Editor

4. THEORY:

In large-scale applications, inefficient state access and frequent re-renders can significantly degrade performance. Redux selectors are used to extract specific pieces of state; however, without optimization, they may recompute values unnecessarily.
Memoization is a technique used to cache computed results and return them only when input data changes. Libraries like Reselect provide memoized selectors that prevent redundant computations and reduce rendering overhead.

Derived state refers to data computed from the existing state rather than stored directly. By using selectors, applications can compute complex data efficiently without duplicating state.

Optimizing selectors and rendering behavior ensures that components update only when necessary, improving responsiveness and scalability.

5. IMPLEMENTATION/PROCEDURE:

1.Define basic selectors for accessing state
2.Implement memoized selectors using createSelector
3.Compute derived data (e.g., filtered posts, grouped data)
4.Integrate selectors into components
5.Use React.memo and useMemo where necessary
6.Analyze and minimize unnecessary re-renders

6. CODE :

```
import React from "react";
import "./App.css";

import { Provider, useDispatch, useSelector } from "react-redux";
import { configureStore, createSlice } from "@reduxjs/toolkit";

import { FaLinkedin, FaInstagram, FaFacebook, FaYoutube, FaWhatsapp } from "react-icons/fa";
import { FaXTwitter } from "react-icons/fa6";

const platformData = {
  LinkedIn: {
    icon: <FaLinkedin />,
    color: "#0A66C2",
    limit: 3000,
  },

  "X (Twitter)": {
    icon: <FaXTwitter />,
    color: "#000000",
    limit: 280,
  },

  Instagram: {
    icon: <FaInstagram />,
    color: "#E1306C",
    limit: 2200,
  },

  Facebook: {
    icon: <FaFacebook />,
    color: "#1877F2",
    limit: 63206,
  },

  YouTube: {
    icon: <FaYoutube />,
    color: "#FF0000",
    limit: 5000,
  },

  WhatsApp: {
    icon: <FaWhatsapp />,
    color: "#25D366",
    limit: 700,
  },
};

const postSlice = createSlice({
  name: "post",

  initialState: {
    platform: "LinkedIn",
    post: "",
    savedPosts: [],
```

```
    likes: 0,
  },

  reducers: {
    setPlatform: (state, action) => {
      state.platform = action.payload;
    },

    setPost: (state, action) => {
      state.post = action.payload;
    },

    savePost: (state) => {
      if (state.post.trim() !== "") {
        state.savedPosts.push({
          platform: state.platform,
          content: state.post,
          date: new Date().toLocaleString(),
        });

        state.post = "";
      }
    },

    clearPost: (state) => {
      state.post = "";
    },

    likePost: (state) => {
      state.likes += 1;
    },
  },
});

const {
  setPlatform,
  setPost,
  savePost,
  clearPost,
  likePost,
} = postSlice.actions;

const store = configureStore({
  reducer: {
    post: postSlice.reducer,
  },
});

function PostComposer() {
  const dispatch = useDispatch();

  const { platform, post, savedPosts, likes } = useSelector(
    (state) => state.post
  );

  const limit = platformData[platform].limit;
```

```
const exceeded = post.length > limit;

return (
  <div className="container">
    <div className="card">

      <h1 style={{ color: platformData[platform].color }}>
        {platformData[platform].icon}
        <br />
        Redux Toolkit Post Composer
      </h1>

      <label>
        <b>Select Platform</b>
      </label>

      <br />
      <br />

      <select
        value={platform}
        onChange={(e) => dispatch(setPlatform(e.target.value))}
      >
        {Object.keys(platformData).map((item) => (
          <option key={item}>{item}</option>
        ))}
      </select>

      <br />
      <br />

      <label>
        <b>Write Your Post</b>
      </label>

      <br />
      <br />

      <textarea
        rows="7"
        value={post}
        placeholder="Write your post..."
        onChange={(e) => dispatch(setPost(e.target.value))}
      />

      <p className={exceeded ? "red" : "green"}>
        Characters: {post.length} / {limit}
      </p>

      {exceeded ? (
        <p className="red">Character Limit Exceeded</p>
      ) : (
        <p className="green">Ready to Post</p>
      )}
```

```
          <div className="buttons">
            <button
              className="save"
              onClick={() => dispatch(savePost())}
            >
              💾 Save Post
            </button>

            <button
              className="clear"
              onClick={() => dispatch(clearPost())}
            >
              🗑 Clear
            </button>

            <button
              className="like"
              onClick={() => dispatch(likePost())}
            >
              ❤️ Like
            </button>
          </div>

          <h3>Total Likes : {likes}</h3>

          <hr />

          <h2>Saved Posts</h2>

          {savedPosts.length === 0 ? (
            <p>No Posts Saved Yet</p>
          ) : (
            savedPosts.map((item, index) => (
              <div key={index} className="postCard">
                <h3 style={{ color: platformData[item.platform].color }}>
                  {platformData[item.platform].icon} {item.platform}
                </h3>

                <p>{item.content}</p>

                <small>{item.date}</small>

                <hr />
              </div>
            ))
          )}
        </div>
      </div>
    );
  }
  function App() {
    return (
      <Provider store={store}>
        <PostComposer />
      </Provider>
    );
```

```
}

export default App;
```

7. OUTPUT :

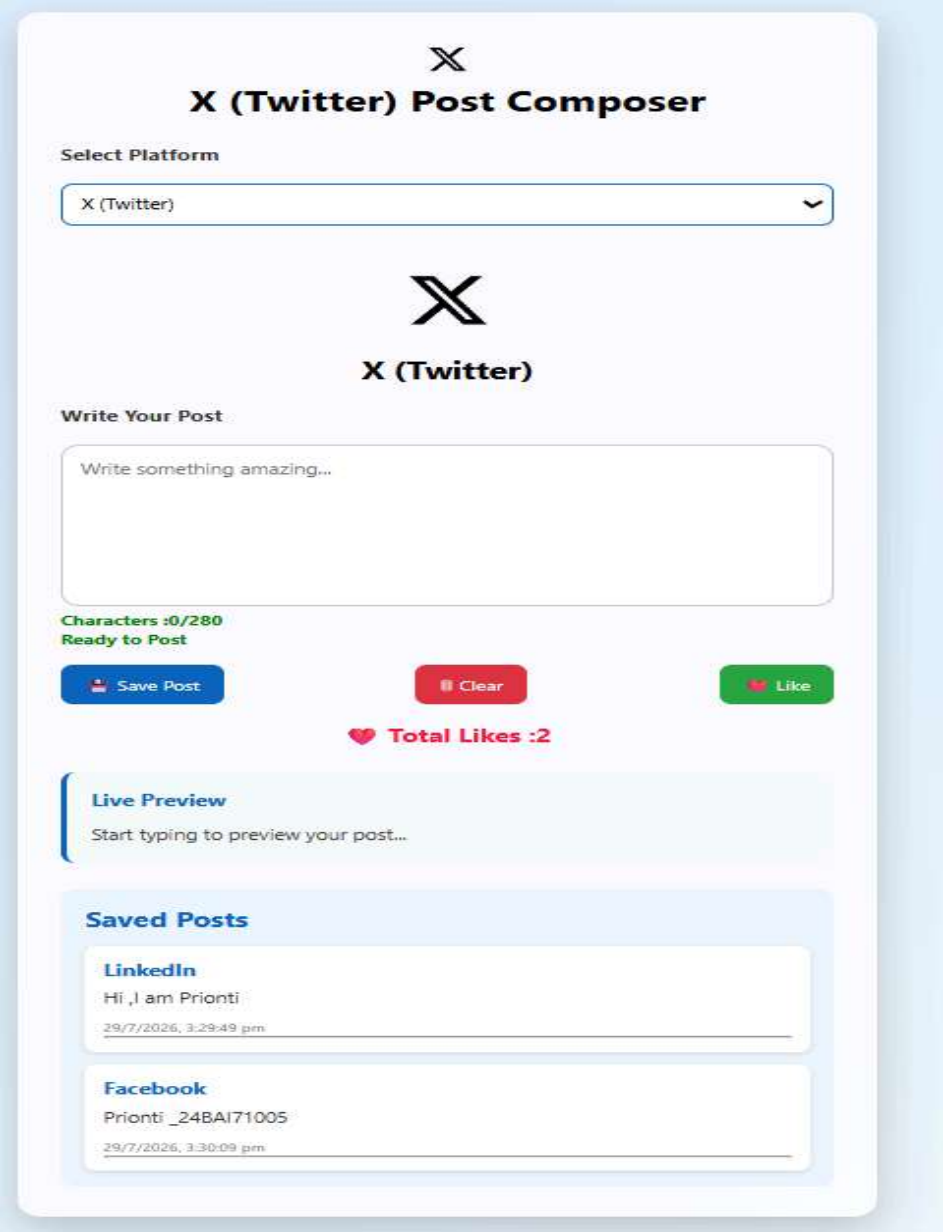


Fig 2.2:Post Composer with Platform Selection and State Management

The output displays a **Redux Toolkit-based Social Media Post Composer** developed using React. Users can select a social media platform, write a post, and view the live character count with platform-specific validation. The **Save Post** button stores the post, the **Clear** button removes the current text, and the **Like** button updates the total likes. All saved posts are displayed along with their platform name and date, demonstrating **centralized state management** using Redux Toolkit.

8. LEARNING OUTCOME:

- Understand the concept of centralized state management using Redux Toolkit.
- Learn to create and manage application state using Redux slices and reducers.
- Implement state updates using Redux actions and the `useDispatch()` hook.
- Access application state efficiently using the `useSelector()` hook.
- Develop a React application for creating, saving, clearing, and liking social media posts.
- Manage platform-specific data such as character limits and platform selection.
- Improve application performance by reducing unnecessary state updates and re-renders.
- Gain practical experience in integrating React with Redux Toolkit for scalable application development.